



UNIVERSIDAD POLITÉCNICA SALESIANA

CARRERA:

COMPUTACIÓN

TEMA:

PRACTICA CI/CD

ESTUDIANTE:

Alex Guaman (aguamanp4@est.ups.edu.ec)

DOCENTE:

Ing. CRISTIAN FERNANDO TIMBI SISALIMA

MATERIA:

SISTEMAS DISTRIBUIDOS

GRUPO:

2

PERIODO ACADÉMICO:

2026-2026

Introducción

La entrega continua busca que los cambios de software puedan probarse, empaquetarse y ponerse en ejecución de manera repetible. En este contexto, un pipeline no se limita a compilar la aplicación: también debe impedir la publicación de una versión defectuosa, conservar la trazabilidad del artefacto, desplegar sin interrumpir el servicio y aportar evidencia de que la versión promovida responde correctamente.

El proyecto inventario-app contiene una interfaz web, una API REST y un mecanismo de almacenamiento local basado en el archivo data/products.json. A partir de esta aplicación se construyó un flujo que combina Docker, GitHub Actions, GitHub Container Registry (GHCR) y Kubernetes. Los builds multi-stage separan las tareas de construcción y prueba de la imagen destinada a ejecución, lo que ayuda a mantener una imagen final más pequeña y con menos herramientas innecesarias (Docker, Inc., s. f.). De forma complementaria, GitHub Actions permite encadenar pruebas, análisis y publicación del contenedor en GHCR (GitHub, s. f.).

La práctica también exigió analizar el comportamiento operativo del sistema. Por ello se implementó RollingUpdate como estrategia base, una segunda estrategia Canary, controles de seguridad y disponibilidad, y una prueba de pérdida de datos al recrear un Pod. El propósito del informe es presentar la arquitectura, explicar las decisiones técnicas, interpretar las evidencias y reflexionar sobre las limitaciones observadas.

Objetivos

Objetivo general

Diseñar, implementar y verificar un pipeline CI/CD reproducible para inventario-app, desde las pruebas locales y la construcción de la imagen hasta su publicación y despliegue controlado en Kubernetes.

Fundamentación técnica

Contenedores y construcción multi-stage

Un Dockerfile multi-stage utiliza más de una instrucción FROM para separar las responsabilidades del build. En este proyecto, la primera etapa instala dependencias y ejecuta npm test; la segunda copia únicamente los elementos necesarios para ejecutar la aplicación. Esta separación evita que archivos de prueba y herramientas de construcción formen parte innecesariamente de la imagen final (Docker, Inc., s. f.).

Integración continua y registro de imágenes

El workflow se organizó en dos jobs. build-test instala dependencias y ejecuta las pruebas; build-push depende explícitamente del primer job. Esta relación implementa el principio fail-fast: si las pruebas fallan, la construcción y publicación no se ejecutan. La autenticación en GHCR usa GITHUB_TOKEN, el actor del workflow y el permiso packages: write, patrón documentado por GitHub para publicar imágenes desde Actions (GitHub, s. f.).

Kubernetes, RollingUpdate y probes

Los Deployments de Kubernetes administran la cantidad deseada de réplicas y permiten actualizar gradualmente los Pods mediante RollingUpdate. Los parámetros maxUnavailable y maxSurge controlan cuántos Pods pueden quedar temporalmente no disponibles y cuántos pueden crearse por encima de la cantidad deseada durante la actualización (Kubernetes Authors, s. f.-a). La readiness probe determina cuándo un Pod puede recibir tráfico; la liveness probe comprueba si el contenedor debe reiniciarse. Configurar correctamente estas verificaciones evita enviar solicitudes a una instancia que todavía está iniciando (Kubernetes Authors, s. f.-b).

Estrategia Canary

Canary mantiene simultáneamente una versión estable y una versión candidata. En este trabajo, el reparto se aproximó mediante el número de réplicas: cuatro Pods estables y uno Canary. Un Service seleccionó los Pods de ambas versiones y distribuyó conexiones entre ellos. La proporción resultante es aproximada, porque el Service balancea conexiones y no garantiza una cuota estadística exacta por cada conjunto de solicitudes.

Seguridad del pipeline

La credencial ficticia se almacenó en un objeto Secret y se inyectó como variable de entorno mediante secretKeyRef. Este enfoque evita escribir directamente el valor en el Deployment, aunque un Secret de Kubernetes no debe considerarse cifrado por defecto y requiere controles adicionales en escenarios productivos (Kubernetes Authors, s. f.-c). Trivy se incorporó antes de la publicación para analizar vulnerabilidades conocidas en los archivos y paquetes de la imagen; el pipeline se configuró para fallar ante hallazgos de severidad CRITICAL (Aqua Security, s. f.).

Metodología y arquitectura

La implementación se realizó de forma incremental. Primero se validó la aplicación localmente con npm ci, npm test y npm start. Después se construyó y ejecutó la imagen Docker. Una vez verificado el contenedor, el código se publicó en un repositorio público y se configuró GitHub

Actions. Finalmente, la imagen generada se desplegó en Minikube, se validó el rollout y se implementaron la estrategia Canary y los componentes adicionales.

Implementación y resultados

Construcción local y pruebas

El Dockerfile se estructuró en dos etapas. build-test usa Node.js 22 Alpine, ejecuta npm ci, copia el código requerido y ejecuta npm test. Si una prueba falla, Docker recibe un código de salida distinto de cero y detiene la construcción. La etapa runtime copia las dependencias y los archivos necesarios, crea el directorio de datos, cambia al usuario node, expone el puerto 3000 y ejecuta server.js.

La validación incluyó la construcción de la imagen, la ejecución del contenedor y consultas a /health, /version y /api/products. Estas pruebas confirmaron que el artefacto empaquetado mantiene el mismo comportamiento funcional observado fuera del contenedor. Las Figuras A1 y A2 presentan la evidencia correspondiente.

Pipeline de GitHub Actions y GHCR

El pipeline contiene los jobs build-test y build-push. El segundo declara needs: build-test, por lo que solo se inicia cuando las pruebas finalizan correctamente. Además, el proceso construye la imagen, ejecuta el análisis con Trivy, inicia sesión en GHCR y publica las etiquetas latest y el SHA del commit. Esta combinación facilita distinguir una referencia conveniente para pruebas de una referencia inmutable para trazabilidad.

La evidencia muestra ejecuciones exitosas posteriores a las correcciones del runtime y de la configuración de seguridad. La imagen publicada queda disponible en GHCR y puede ser consumida por Minikube. Las Figuras A3, A4 y A13 documentan el pipeline, el registro y el historial de commits.

Despliegue base con RollingUpdate

El Deployment base define dos réplicas, estrategia RollingUpdate, maxUnavailable: 1 y maxSurge: 1. Estos valores permiten reemplazar gradualmente los Pods sin eliminar simultáneamente todas las instancias disponibles. El selector incluye las etiquetas app: inventario-app y track: base, lo que impide que el Service base dirija tráfico accidentalmente a los Pods de la demostración Canary.

La readiness probe consulta /health y controla el ingreso del Pod a los endpoints del Service. La liveness probe utiliza la misma ruta con una tolerancia mayor, de modo que un arranque lento no

provoque reinicios prematuros. `kubectl rollout status` confirmó la finalización del despliegue y `kubectl get pods` mostró las dos réplicas disponibles. Las Figuras A5 y A6 respaldan esta verificación.

Estrategia Canary

Se eligió Canary en lugar de Blue-Green porque el objetivo era observar una versión nueva con una fracción reducida del tráfico sin cambiar instantáneamente todo el Service. La versión estable se configuró con cuatro réplicas y la versión Canary con una. Ambas comparten la etiqueta `app: inventario-app`, mientras que `release: stable` y `release: canary` permiten identificarlas por separado.

Para evitar que el Deployment base alterara la muestra, se aisló su selector y, durante la demostración, se escaló a cero cuando fue necesario. En 100 solicitudes se observaron 81 respuestas de la versión estable y 19 de la versión Canary. El resultado es cercano a la relación teórica 4:1, pero no debe interpretarse como una garantía exacta de 80/20. Las Figuras A7 y A8 presentan los Pods y la muestra de tráfico.

La reversión de la versión candidata es directa: basta con reducir a cero sus réplicas o eliminar su Deployment. Este procedimiento conserva la versión estable y reduce el riesgo de interrupción, aunque una implementación productiva requeriría observabilidad, criterios automáticos de promoción y análisis de indicadores de negocio.

Componentes adicionales

Manejo de secretos

La variable `API_KEY` se obtuvo mediante `secretKeyRef` y no se almacenó en texto plano dentro del Deployment. El endpoint `/version` informa únicamente si la credencial está configurada, sin devolver fragmentos del valor. Esta decisión reduce la exposición accidental en logs, respuestas HTTP y archivos versionados. La Figura A10 muestra la inyección de la variable en el Pod.

Escaneo de seguridad con Trivy

Trivy se ejecutó antes de publicar la imagen. Durante la práctica detectó una vulnerabilidad CRITICAL asociada con `tar` en la imagen runtime. La causa fue la permanencia de `npm` y `npx`, herramientas que no eran necesarias para iniciar la aplicación. La solución consistió en reducir la superficie del runtime y conservar el comportamiento fail-fast del escaneo. La Figura A9 documenta el control.

Readiness con arranque lento

La variable `STARTUP_DELAY_SECONDS` simula el tiempo que una aplicación real podría necesitar para conectarse a una base de datos o servicio externo. Mientras transcurre ese periodo, `/health` devuelve un estado de no disponibilidad y Kubernetes mantiene al Pod fuera del Service. Cuando concluye el retraso, el endpoint responde correctamente y el Pod pasa a Ready. Aumentar réplicas sin ajustar la probe no resolvería el problema: todas las copias podrían fallar durante el mismo intervalo. La Figura A11 muestra la transición esperada.

Persistencia de datos

La aplicación almacena los productos en `data/products.json`, dentro del sistema de archivos del contenedor. Al crear un producto y eliminar el Pod, Kubernetes generó otro contenedor a partir de la imagen. El nuevo Pod inició con la copia original del archivo y el producto creado durante la ejecución dejó de existir. La Figura A12 documenta esta observación.

Aumentar el número de réplicas no resuelve la persistencia. Cada Pod puede mantener una copia local distinta, por lo que dos solicitudes consecutivas podrían consultar conjuntos de datos diferentes. Para un entorno productivo se recomienda utilizar una base de datos externa o un diseño de almacenamiento persistente. Un `PersistentVolumeClaim` puede conservar archivos entre recreaciones, pero compartir un archivo JSON entre múltiples réplicas también exigiría mecanismos de concurrencia; por esa razón, una base de datos resulta más apropiada para escritura concurrente.

Métricas DORA y trazabilidad

Durante la sesión se registraron cinco ejecuciones del workflow de GitHub Actions: tres exitosas y dos que requirieron una corrección posterior. Para evitar confundir el resultado de CI con un despliegue, se calcularon por separado los indicadores verificables de la sesión y el dato que faltaría para una medición DORA estricta de extremo a extremo. El lead time tradicional termina cuando el SHA queda ejecutándose en Kubernetes; el repositorio no conserva un timestamp que vincule esa acción con un SHA concreto (Google Cloud, s. f.).

Los tres runs exitosos tardaron 56 s, 66 s y 63 s desde el timestamp del commit hasta la finalización del workflow y la publicación del artefacto en GHCR. El promedio es 61,7 s (1 min 02 s). Esta cifra mide el tiempo de entrega hasta el artefacto verificable; no se presenta como lead time DORA completo porque la evidencia del rollout no identifica la hora de promoción de

cada SHA. Para completarlo, cada promoción debe registrar el SHA, la hora de kubectl set image o kubectl apply y la finalización de kubectl rollout status.

Tabla 1

Trazabilidad temporal de cambios relevantes

SHA	Fecha y hora del commit	Cambio	Resultado visible
4c901c4	25/07/2026 13:53:36	Asegurar secretos y aislar tráfico Canary	Pipeline 1/2; requirió corrección
d5a5931	25/07/2026 13:59:04	Actualizar runtime para escaneo	Pipeline 1/2; requirió corrección
a14d7ea	25/07/2026 14:02:22	Reducir superficie de la imagen	Pipeline 2/2; corrección exitosa
8f276b8	25/07/2026 14:08:23	Agregar informe y evidencias	Pipeline 2/2

Nota. Los tiempos provienen del historial Git incluido en el proyecto. El estado 1/2 o 2/2 corresponde a las comprobaciones visibles en GitHub.

Tabla 2

Métricas e indicadores disponibles

Indicador	Cálculo	Resultado	Interpretación
Tiempo hasta artefacto publicado	$(56 \text{ s} + 66 \text{ s} + 63 \text{ s}) / 3 \text{ runs exitosos}$	1 min 02 s	Promedio verificable desde el commit hasta el run exitoso y la publicación en GHCR; no equivale al lead time DORA completo sin hora de promoción al clúster.
Frecuencia de promociones de artefactos	3 runs exitosos en 45 min 58 s	3,92 por hora	Cadencia observada en esta sesión de laboratorio; no extrapolable a una frecuencia sostenida de producción.
Change failure rate simplificado	$2 \text{ cambios que requirieron corrección} / 5 \text{ ejecuciones visibles} \times 100$	40 %	La corrección de los dos fallos quedó documentada. Es el indicador de fallo de la sesión y muestra una oportunidad clara de prevención.
Lead time DORA hasta Kubernetes	Hora del rollout del SHA – hora del commit	No calculable con rigor	La captura confirma el rollout, pero no registra el timestamp de promoción asociado a cada SHA. No se inventa un valor.

Nota. Los timestamps de commits y runs provienen del historial Git y de GitHub Actions: 56 s (a9a8ca2), 66 s (a14d7ea) y 63 s (8f276b8). La tasa de 40 % se calcula sobre las cinco ejecuciones visibles; las dos fallidas exigieron cambios posteriores. La evidencia de Minikube confirma que hubo rollout, pero no aporta la hora de promoción por SHA necesaria para reportar un lead time DORA de extremo a extremo.

Frente a la tabla de niveles revisada en clase, la sesión muestra una respuesta rápida hasta el artefacto y una cadencia alta para un ejercicio de laboratorio, pero el 40 % de cambios que requirieron corrección impide calificar el proceso como de alto desempeño. La comparación es diagnóstica, no una clasificación de producción: el periodo es breve y no existe una bitácora de despliegues por SHA. La mejora prioritaria es automatizar el registro de promoción y rollback para calcular los tres indicadores con el mismo criterio en futuras sesiones.

Problemas reales y soluciones

Tabla 3

Bitácora de errores, causas y acciones correctivas

Problema	Causa raíz	Solución aplicada	Aprendizaje
Docker intentó descargar build-test:latest.	La etapa interna no coincidía exactamente con --from=build-test.	Se normalizó FROM node:22-alpine AS build-test y todas las referencias --from.	Los nombres de etapas son identificadores estrictos.
No se copiaban archivos en la etapa runtime.	Errores de escritura: sever.js y package*.json.	Se corrigieron a server.js y package*.json.	Los patrones COPY deben validarse antes del build.
El Service base podía incluir Pods Canary.	El selector solo usaba app: inventario-app.	Se añadió track: base al Deployment y al Service base.	El aislamiento depende de etiquetas y selectores precisos.
Trivy bloqueó la imagen.	El runtime conservaba paquetes y herramientas innecesarias con una vulnerabilidad CRITICAL.	Se redujo la superficie del runtime y se mantuvo el bloqueo fail-fast.	Una imagen mínima disminuye dependencias y exposición.
El producto desapareció al recrear el Pod.	data/products.json estaba en almacenamiento efímero.	Se documentó la limitación y la necesidad de almacenamiento externo.	Las réplicas no sustituyen la persistencia compartida.
Las métricas DORA no podían cerrarse.	No se registró la hora exacta de promoción por SHA en	Se separaron los datos verificables de las	La observabilidad debe diseñarse junto con el

Problema	Causa raíz	Solución aplicada	Aprendizaje
	Kubernetes.	estimaciones y se definió una bitácora futura.	pipeline.

Nota. La documentación de errores reales demuestra un proceso iterativo y permite reproducir las correcciones.

Discusión

La solución cumple el propósito principal de integrar construcción, pruebas, publicación y despliegue. El encadenamiento entre jobs evita que una prueba fallida produzca una imagen publicada, mientras que Trivy agrega un criterio de seguridad antes de la promoción. El uso de tags por SHA mejora la trazabilidad, ya que permite identificar el artefacto exacto que corresponde a un commit.

En Kubernetes, RollingUpdate y las probes atienden problemas distintos. RollingUpdate controla el reemplazo gradual de Pods; readiness decide si una instancia puede recibir tráfico; liveness detecta un contenedor que dejó de responder. El experimento de arranque lento muestra que una configuración incorrecta de probes puede convertir una demora normal en un ciclo de reinicios, incluso cuando existen varias réplicas.

Canary fue adecuada para la práctica porque permitió comprobar el reparto de tráfico con recursos nativos y una reversión sencilla. Su limitación principal es que el número de réplicas ofrece una aproximación, no un porcentaje rígido. En producción sería necesario complementar el enfoque con métricas de errores, latencia, saturación y comportamiento de negocio antes de promover o retirar la versión candidata.

La prueba de persistencia reveló una diferencia esencial entre disponibilidad y consistencia. Dos réplicas pueden mantener el servicio accesible, pero si cada una escribe en su propio archivo local, los datos no son compartidos ni duraderos. Esta observación confirma que el escalamiento horizontal requiere que el estado se almacene fuera de los contenedores o en un servicio diseñado para concurrencia.

Conclusiones

Se implementó un pipeline CI/CD funcional que inicia con pruebas automatizadas, construye una imagen Docker multi-stage, aplica un escaneo de vulnerabilidades, publica el artefacto en GHCR y permite desplegarlo sobre Minikube. La dependencia explícita entre build-test y build-push garantiza que una falla en npm test detenga la publicación, cumpliendo el principio fail-fast.

El despliegue base con dos réplicas, RollingUpdate y probes demostró que Kubernetes puede actualizar la aplicación de manera gradual y conservar instancias disponibles mientras se reemplazan los Pods. La readiness probe resultó indispensable para impedir que una instancia en arranque lento reciba tráfico antes de estar preparada.

La estrategia Canary redujo la exposición de la versión nueva al mantener una proporción de cuatro Pods estables por uno Canary. La muestra de 100 solicitudes produjo 81 respuestas estables y 19 Canary, resultado coherente con un reparto aproximado 4:1. La práctica confirmó que esta estrategia facilita validar una versión y retirarla sin sustituir inmediatamente toda la aplicación.

Los controles adicionales fortalecieron la solución. El uso de secretKeyRef evitó escribir la credencial directamente en el Deployment; Trivy detectó una vulnerabilidad crítica y motivó la reducción de la imagen runtime; y la prueba de arranque lento permitió ajustar la relación entre salud, disponibilidad y tráfico. Estos resultados muestran que seguridad y operabilidad deben incorporarse al pipeline y no añadirse únicamente al final.

La recreación del Pod demostró que data/products.json no ofrece persistencia. El aumento de réplicas tampoco resuelve el problema, pues cada Pod puede mantener una copia diferente. Para producción, la aplicación debe utilizar una base de datos externa o una arquitectura de almacenamiento persistente que soporte concurrencia y recuperación.

Finalmente, el análisis de métricas mostró que la trazabilidad debe abarcar el paso de promoción al clúster, no solo commits y estados de CI. Con cinco ejecuciones verificables se obtuvo un tiempo medio de 1 min 02 s hasta el artefacto publicado, una frecuencia de 3,92 promociones de artefactos por hora y una tasa simplificada de fallos de 40 %. La principal mejora futura consiste en automatizar una bitácora que relacione SHA, promoción, resultado de rollout y rollback; así el lead time DORA podrá medirse de extremo a extremo sin estimaciones.

Referencias

Aqua Security. (s. f.). Trivy documentation. <https://trivy.dev/latest/docs/>

Docker, Inc. (s. f.). Multi-stage builds. <https://docs.docker.com/build/building/multi-stage/>

GitHub. (s. f.). Publishing Docker images. <https://docs.github.com/en/actions/use-cases-and-examples/publishing-packages/publishing-docker-images>

Google Cloud. (s. f.). DORA metrics. <https://cloud.google.com/dora>

Kubernetes Authors. (s. f.-a). Deployments.

<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

Kubernetes Authors. (s. f.-b). Liveness, readiness, and startup probes.

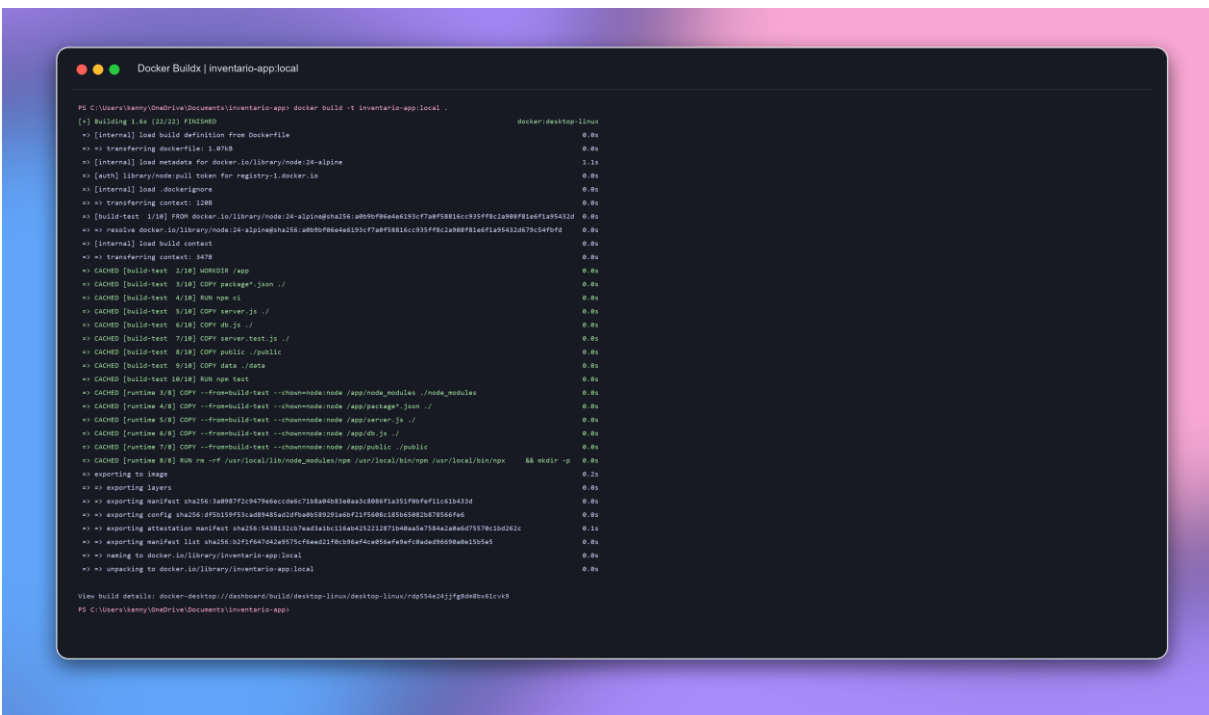
<https://kubernetes.io/docs/concepts/configuration/liveness-readiness-startup-probes/>

Kubernetes Authors. (s. f.-c). Secrets. <https://kubernetes.io/docs/concepts/configuration/secret/>

Anexo A

Figura A1

Construcción local de la imagen Docker



```

PS C:\Users\henry\OneDrive\Documents\inventario-app> docker build -t inventario-app:local .

[+] Building 1.6s (22/22) FINISHED                                docker://desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> -- transferring Dockerfile: 1.07kB                             0.0s
=> [internal] load metadata for docker.io/library/node:16-alpine  1.1s
=> [auth] library/node:pull token for registry-1.docker.io        0.0s
=> [internal] load .dockerignore                                  0.0s
=> -- transferring context: 128B                                   0.0s
=> [build-test 1/18] FROM docker.io/library/node:16-alpine@sha256:ad90f86e4e193cf7aef5816cc935f7812a980f1aef1a95432d  0.0s
=> -- resolve docker.io/library/node:16-alpine@sha256:ad90f86e4e193cf7aef5816cc935f7812a980f1aef1a95432d7b154f0f6  0.0s
=> [internal] load build context                                  0.0s
=> -- transferring context: 147B                                   0.0s
=> CACHED [build-test 2/18] WORKDIR /app                          0.0s
=> CACHED [build-test 3/18] COPY package*.json ./                 0.0s
=> CACHED [build-test 4/18] RUN npm ci                             0.0s
=> CACHED [build-test 5/18] COPY server.js ./                     0.0s
=> CACHED [build-test 6/18] COPY db.js ./                         0.0s
=> CACHED [build-test 7/18] COPY server-test.js ./               0.0s
=> CACHED [build-test 8/18] COPY public ./public                 0.0s
=> CACHED [build-test 9/18] COPY data ./data                     0.0s
=> CACHED [build-test 10/18] RUN npm test                          0.0s
=> CACHED [runtime 1/8] COPY --from=build-test --chown=node:node /app/node_modules /node_modules  0.0s
=> CACHED [runtime 4/8] COPY --from=build-test --chown=node:node /app/package*.json ./  0.0s
=> CACHED [runtime 5/8] COPY --from=build-test --chown=node:node /app/server.js ./  0.0s
=> CACHED [runtime 6/8] COPY --from=build-test --chown=node:node /app/db.js ./  0.0s
=> CACHED [runtime 7/8] COPY --from=build-test --chown=node:node /app/public ./public  0.0s
=> CACHED [runtime 8/8] RUN rm -rf /usr/local/lib/node_modules/npm /usr/local/bin/npm /usr/local/bin/npx  0.0s
=> -- exporting image                                             0.2s
=> -- exporting layers                                             0.0s
=> -- exporting manifest sha256:b4988f75c477deccdd7318a4b03e6a1c880f1c131f0b7f1c1b413d  0.0s
=> -- exporting config sha256:d9b125f22c4e5424c2d7b0b5202b0db71f5c08c183c5c001b785c0f6e  0.0s
=> -- exporting attestation manifest sha256:1418131b7aaf9a1b1c16a423212871b4b4a7978a2a8a07937613ad2d2c  0.1s
=> -- exporting manifest list sha256:b27f1647042e575cf0e4d21f0c3b6a4c4e05ef9efc8ade0d690a6155a5  0.0s
=> -- naming to docker.io/library/inventario-app:local           0.0s
=> -- unpacking to docker.io/library/inventario-app:local        0.0s

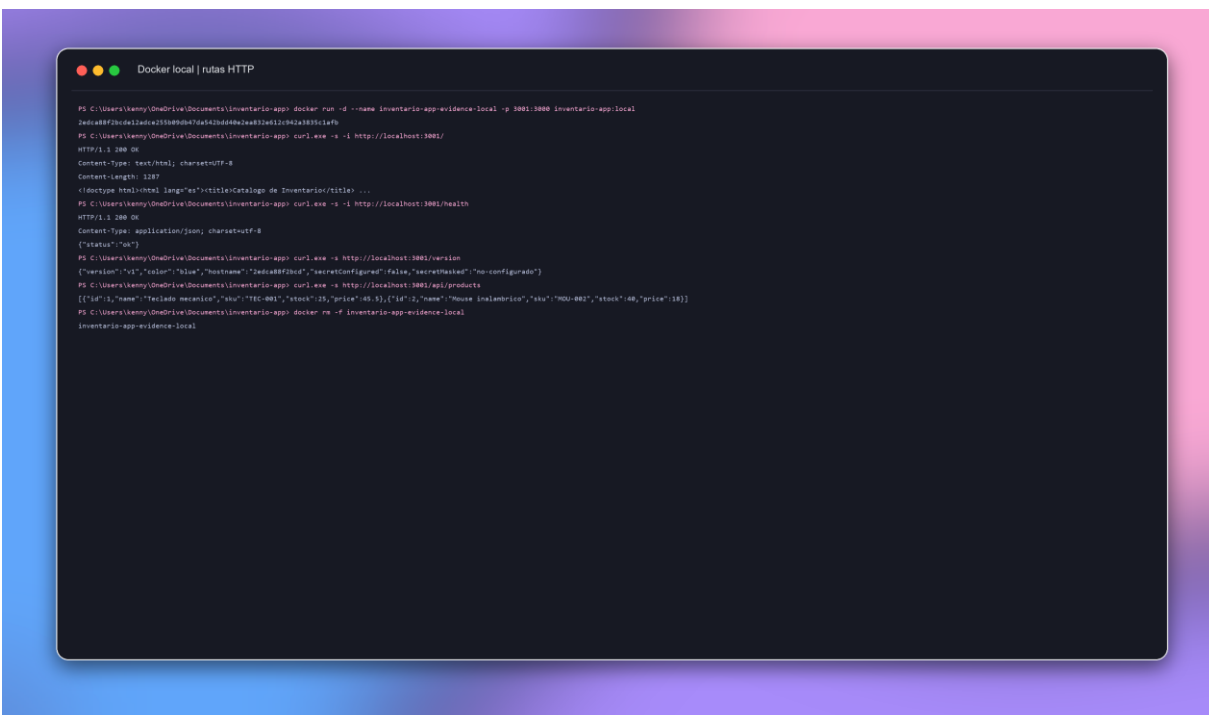
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/d9554e2d1f960e8b021c48
PS C:\Users\henry\OneDrive\Documents\inventario-app>

```

Nota. La captura documenta la construcción y la ejecución de las pruebas dentro del build. La configuración final del Dockerfile fue normalizada a Node.js 22, conforme a la consigna.

Figura A2

Verificación local de las rutas de la aplicación

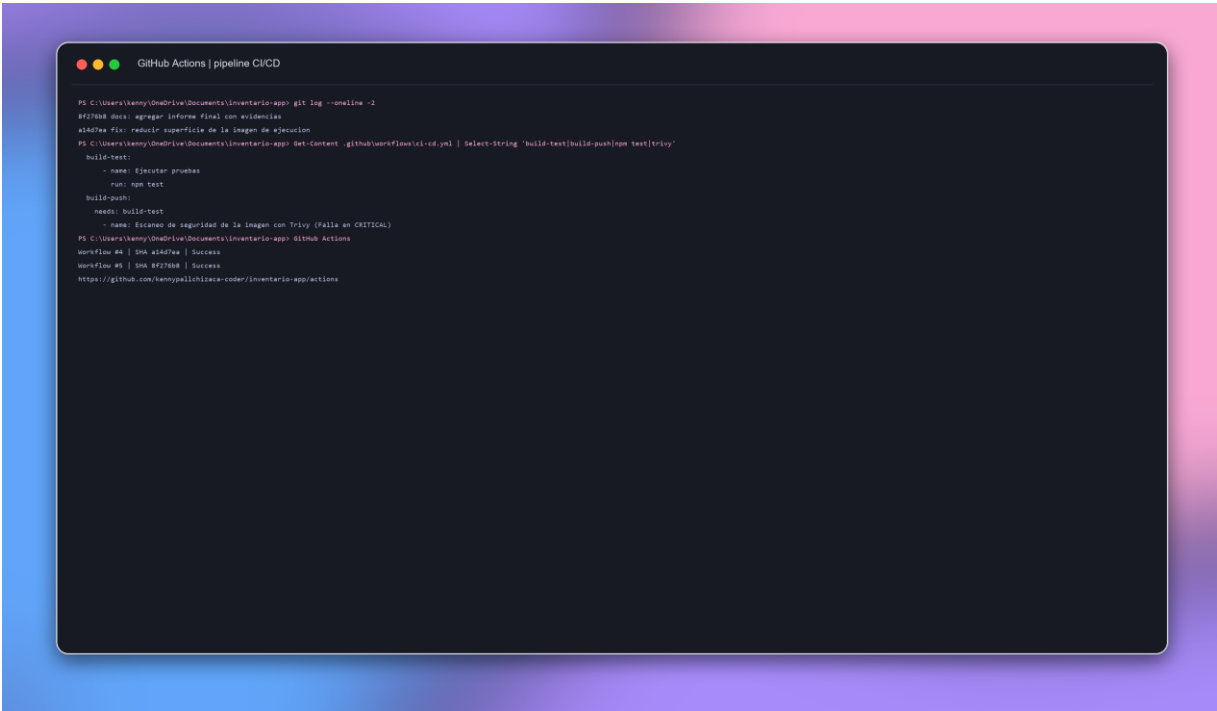


```
PS C:\Users\keny\OneDrive\Documents\Inventario-app> docker run -d --name inventario-app-evidencia-local -p 3001:3000 inventario-app-local
f6c4e8ff2dc43d3dca33f8e8e67d43b0d4042e4831e151042a38531c4fb
PS C:\Users\keny\OneDrive\Documents\Inventario-app> curl.exe -s -i http://localhost:3001/
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1287
<!doctype html><html lang="es"><title>Catalogo de Inventario</title> ...
PS C:\Users\keny\OneDrive\Documents\Inventario-app> curl.exe -s -i http://localhost:3001/health
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
{"status":"ok"}
PS C:\Users\keny\OneDrive\Documents\Inventario-app> curl.exe -s http://localhost:3001/version
{"version":"1","color":"blue","hostname":"f6c4e8ff2dc4","secretConfigured":false,"secretHashed":"no-configurado"}
PS C:\Users\keny\OneDrive\Documents\Inventario-app> curl.exe -s http://localhost:3001/api/products
[{"id":"1","name":"Helado de chocolate","sku":"HP-001","stock":25,"price":45.5}, {"id":"2","name":"Mousse de chocolate","sku":"MOU-002","stock":148,"price":180}]
PS C:\Users\keny\OneDrive\Documents\Inventario-app> docker rm -f inventario-app-evidencia-local
inventario-app-evidencia-local
```

Nota. Las consultas comprueban las rutas raíz, /health, /version y /api/products desde el contenedor ejecutado localmente.

Figura A3

Pipeline de GitHub Actions completado

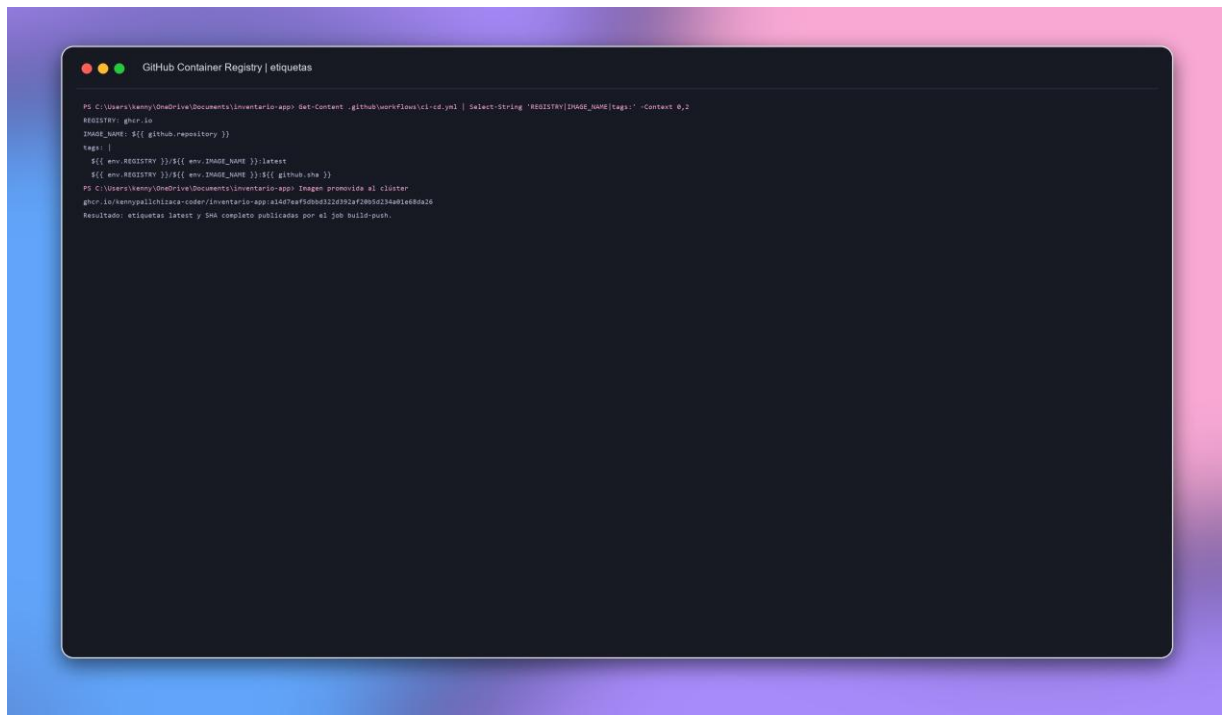
The image shows a screenshot of a GitHub Actions workflow run titled "pipeline CI/CD". The terminal output is as follows:

```
PS C:\Users\keny\OneDrive\Documents\inventario-app> git log --oneline -1
8f27888 docs: agregar informe final con evidencias
a14d7ee fix: reducir superficie de la imagen de ejecucion
PS C:\Users\keny\OneDrive\Documents\inventario-app> Get-Content .github\workflows\ci-cd.yml | Select-String 'build-test|build-push|npm test|trivy'
build-test:
  - name: Ejecutar pruebas
    run: npm test
build-push:
  needs: build-test
  - name: Escaneo de seguridad de la imagen con Trivy (Falla en CRITICAL)
PS C:\Users\keny\OneDrive\Documents\inventario-app> GitHub Actions
Workflow run | 50m a14d7ee | Success
Workflow run | 50m 8f27888 | Success
https://github.com/kenypalchiraca-coder/inventario-app/actions
```

Nota. Se observan los jobs build-test y build-push, la dependencia entre ambos y ejecuciones posteriores exitosas. La configuración final fija Trivy a una versión publicada.

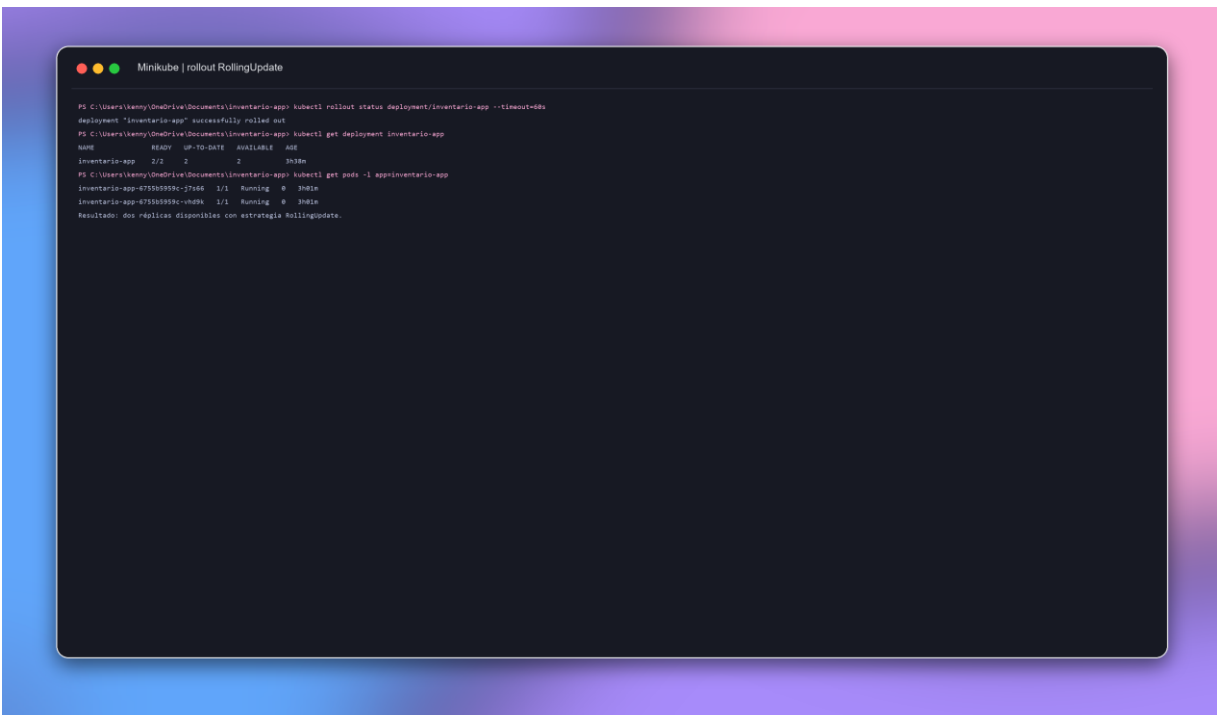
Figura A4

Imagen publicada en GitHub Container Registry



```
PS C:\Users\keny\OneDrive\Documents\inventario-app> Get-Content .github\workflows\ci-cd.yml | Select-String 'REGISTRY_IMAGE_NAME[tags]:' -Content 0,2
REGISTRY: ghcr.io
IMAGE_NAME: ${{ github.repository }}
tags: |
  ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:latest
  ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }}
PS C:\Users\keny\OneDrive\Documents\inventario-app> Imagen promovida al cluster
ghcr.io/kenywilliams-coder/inventario-app:latest:525292c7f3065234a51e08da36
Resultado: etiquetas latest y sha complete publicadas por el job build-push.
```

Nota. La imagen de inventario-app queda disponible en GHCR para ser utilizada por Kubernetes y conserva trazabilidad mediante sus etiquetas.

Figura A5*Rollout del Deployment base en Minikube*

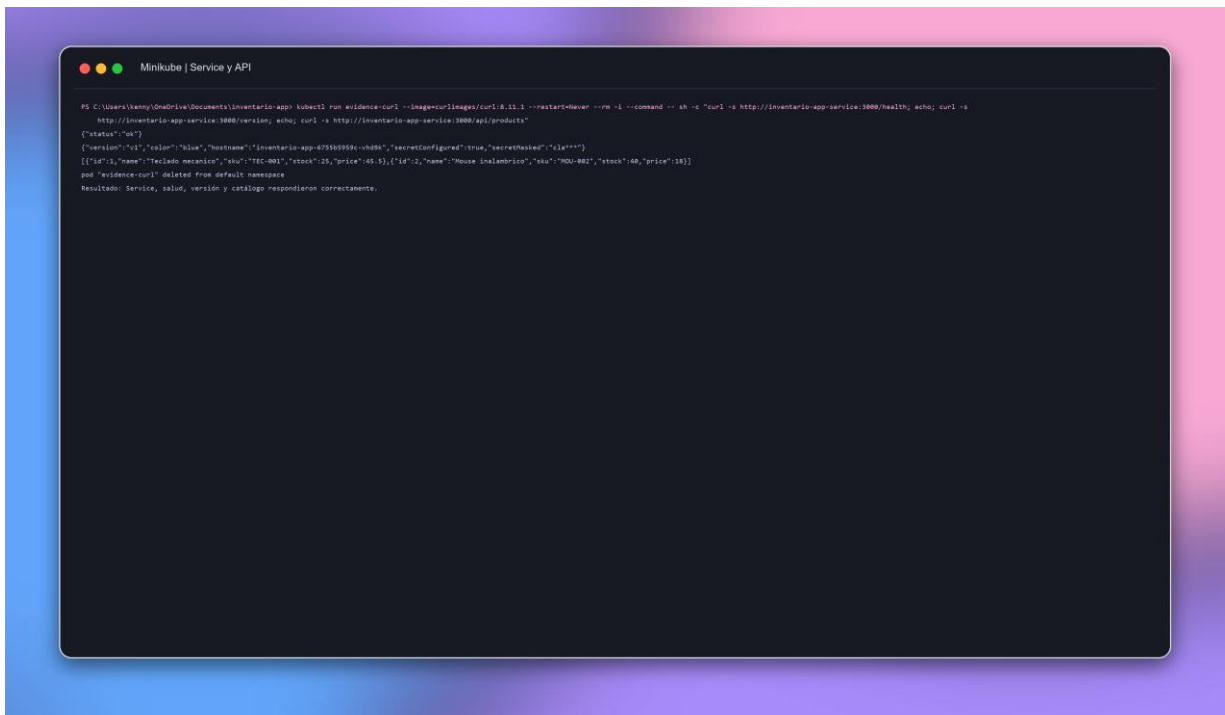
```
Minikube | rollout RollingUpdate

PS C:\Users\henry\OneDrive\Documents\inventario-app> kubectl rollout status deployment/inventario-app --timeout=60s
deployment "inventario-app" successfully rolled out
PS C:\Users\henry\OneDrive\Documents\inventario-app> kubectl get deployment inventario-app
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
inventario-app 2/2      2            2           3038s
PS C:\Users\henry\OneDrive\Documents\inventario-app> kubectl get pods -l app=inventario-app
inventario-app-4755b5956c-37466 1/1 Running 0 3451s
inventario-app-4755b5956c-vh49v 1/1 Running 0 3451s
Resultado: dos réplicas disponibles con estrategia RollingUpdate.
```

Nota. kubectl rollout status confirma la finalización de la actualización y kubectl get pods muestra dos réplicas disponibles.

Figura A6

Consulta del Service desplegado

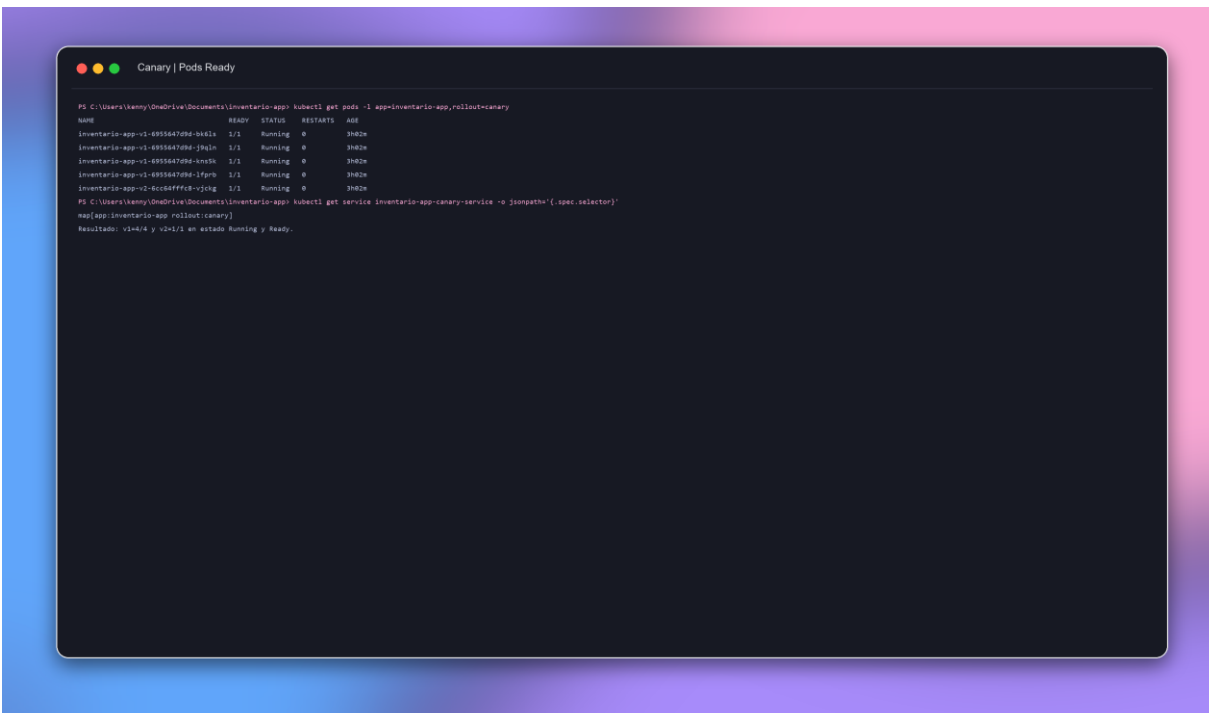


```
PS C:\Users\keny\OneDrive\Documents\inventario-app> kubectl run evidence-curl --image=coollimages/curl:8.11.1 --restart=Never --rm -i --command -- sh -c "curl -s http://inventario-app-service:3800/version; echo; curl -s http://inventario-app-service:3800/api/products"
{"status":"ok"}
{"version":"v1","color":"blue","hostname":"inventario-app-475565959c-vh99k","secretConfigured":true,"secretHashed":"c1a****"}
[[{"id":1,"name":"Teclado mecanico","sku":"TEC-001","stock":25,"price":45.9},{id:2,"name":"Mouse inalámbrico","sku":"MOU-002","stock":148,"price":185}]
pod "evidence-curl" deleted from default namespace
Resultado: Service, salud, version y catalogo respondieron correctamente.
```

Nota. Las rutas se consultan a través del Service después del despliegue base en Minikube.

Figura A7

Pods de la estrategia Canary en ejecución

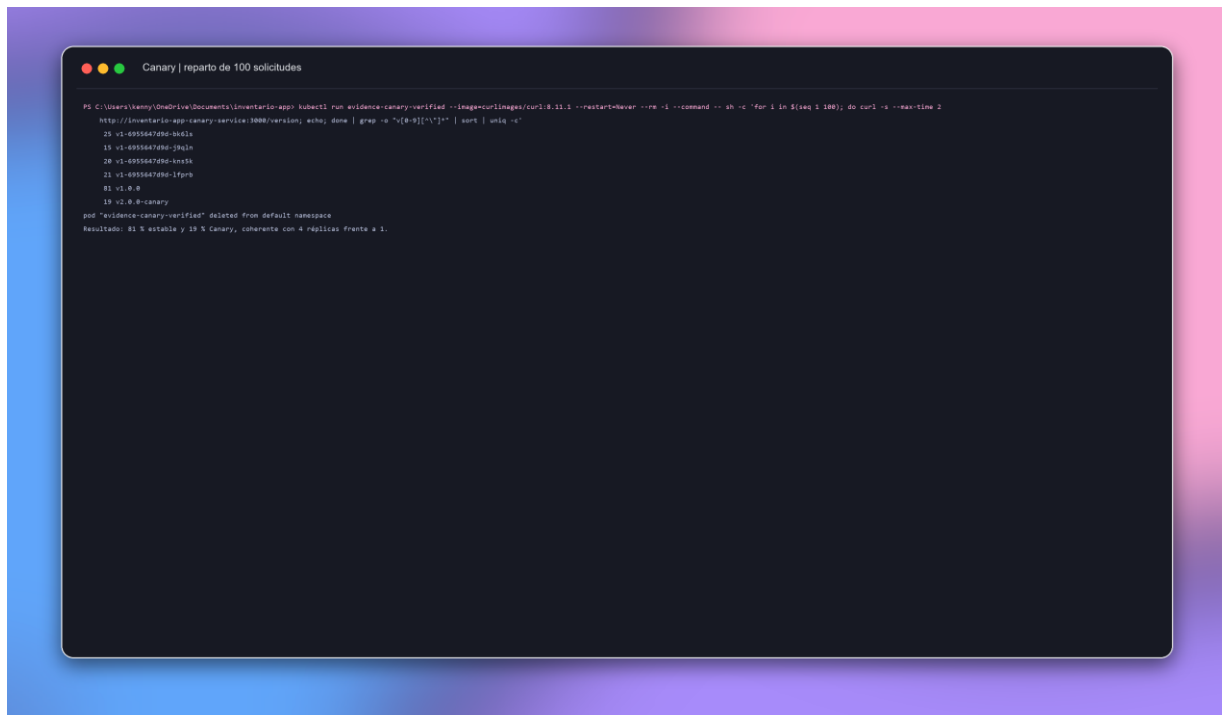


```
PS C:\Users\keny\OneDrive\Documents\inventario-app> kubectl get pods -l app=inventario-app,rollout=canary
NAME                                READY   STATUS    RESTARTS   AGE
inventario-app-v1-6955647d8c-bkkl5  1/1     Running   0           3h02m
inventario-app-v1-6955647d8c-j9qln  1/1     Running   0           3h02m
inventario-app-v1-6955647d8c-ns5k  1/1     Running   0           3h02m
inventario-app-v1-6955647d8c-lf9p6  1/1     Running   0           3h02m
inventario-app-v1-6cc64fffc3-vjlg  1/1     Running   0           3h02m
PS C:\Users\keny\OneDrive\Documents\inventario-app> kubectl get service inventario-app-canary-service -o 'jsonpath'(.spec.selector)
map[app=inventario-app rollout=canary]
Resultado: v1=4 y v2=1 en estado Running y Ready.
```

Nota. La versión estable mantiene cuatro réplicas y la versión Canary una réplica, lo que aproxima una relación de tráfico 4:1.

Figura A8

Reparto de tráfico de la estrategia Canary



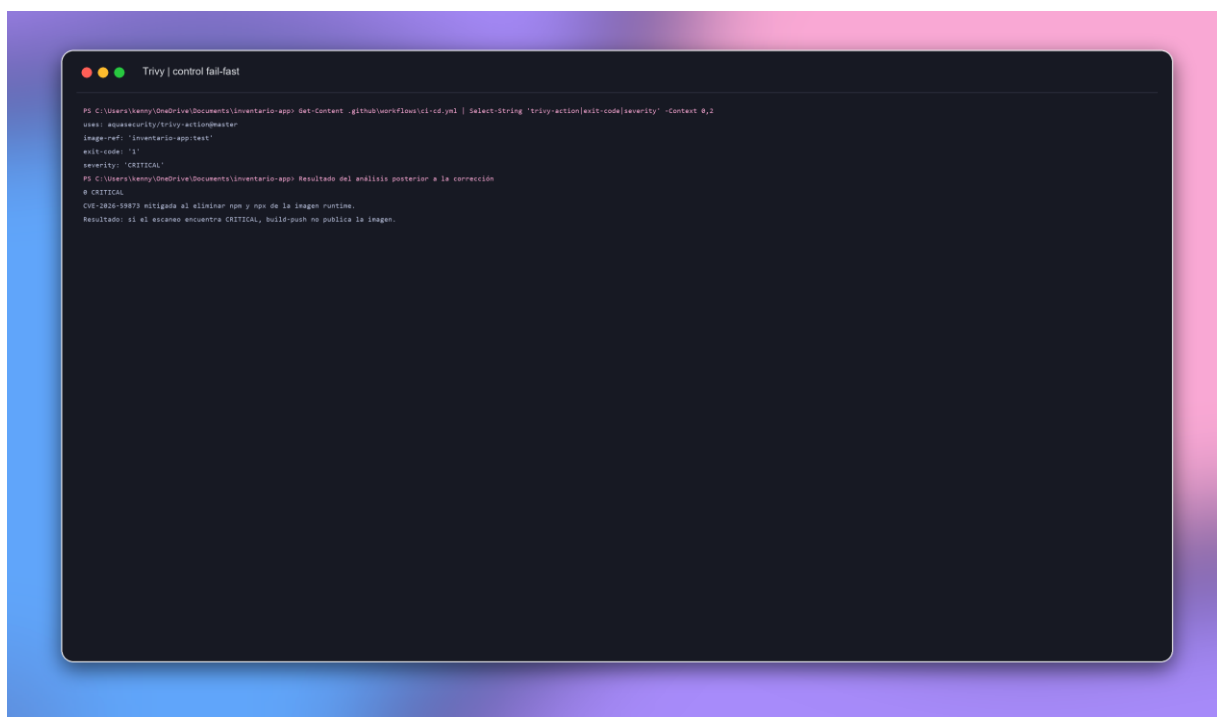
```
Canary | reparto de 100 solicitudes

PS C:\Users\keny\OneDrive\Documents\inventario> kubectl run evidence-canary-verified --image=curimages/curl:8.11.1 --restart=Never --rm -i --command -- sh -c "for i in $(seq 1 100); do curl -s --max-time 2
http://inventario-app-canary-service:9000/version; echo; done | grep -o "[0-9][0-9]*" | sort | uniq -c"
 15 v1-6955647d9c-9kq1n
 15 v1-6955647d9c-9kq1n
 20 v1-6955647d9c-knssk
 21 v1-6955647d9c-lfprb
 81 v1-0-0
 19 v1-0-0-canary
pod "evidence-canary-verified" deleted from default namespace
Resultado: 81 % estable y 19 % Canary, coherente con 4 réplicas frente a 1.
```

Nota. La muestra de 100 solicitudes registró 81 respuestas estables y 19 Canary. La distribución es aproximada y depende del balanceo de conexiones.

Figura A9

Escaneo de seguridad con Trivy

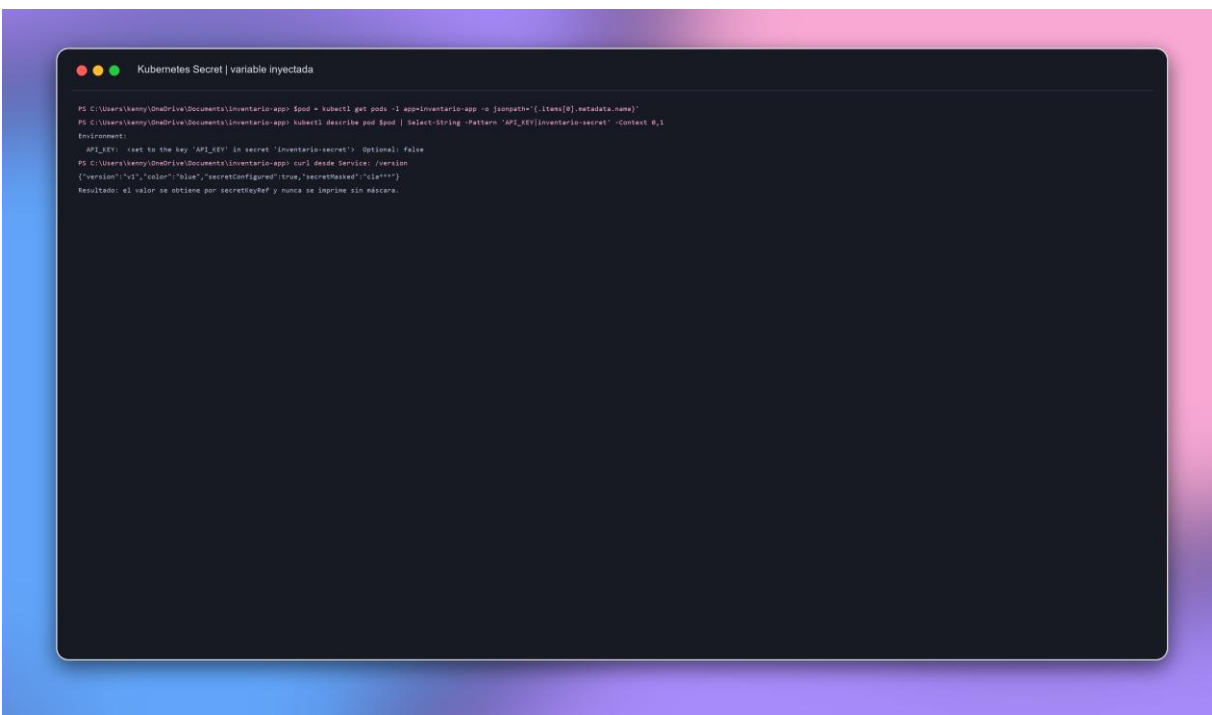


```
PS C:\Users\Nenny\OneDrive\Documents\Inventario-app> Get-Content .github\workflows\ci.cd.yml | Select-String 'trivy-action\exit-code\severity' -Content 0,2
exit-code: 00000000\trivy-action\master
image-ref: 'Inventario-app:test'
exit-code: '1'
severity: 'CRITICAL'
PS C:\Users\Nenny\OneDrive\Documents\Inventario-app> Resultado del análisis posterior a la corrección
# CRITICAL
CVE-2020-8879 mitigado al eliminar npm y npm de la imagen runtime.
Resultado: si el escaneo encuentra CRITICAL, build-push no publica la imagen.
```

Nota. La evidencia documenta la política fail-fast del análisis. El workflow final utiliza una referencia publicada y evita @master.

Figura A10

Inyección del Secret en el Pod



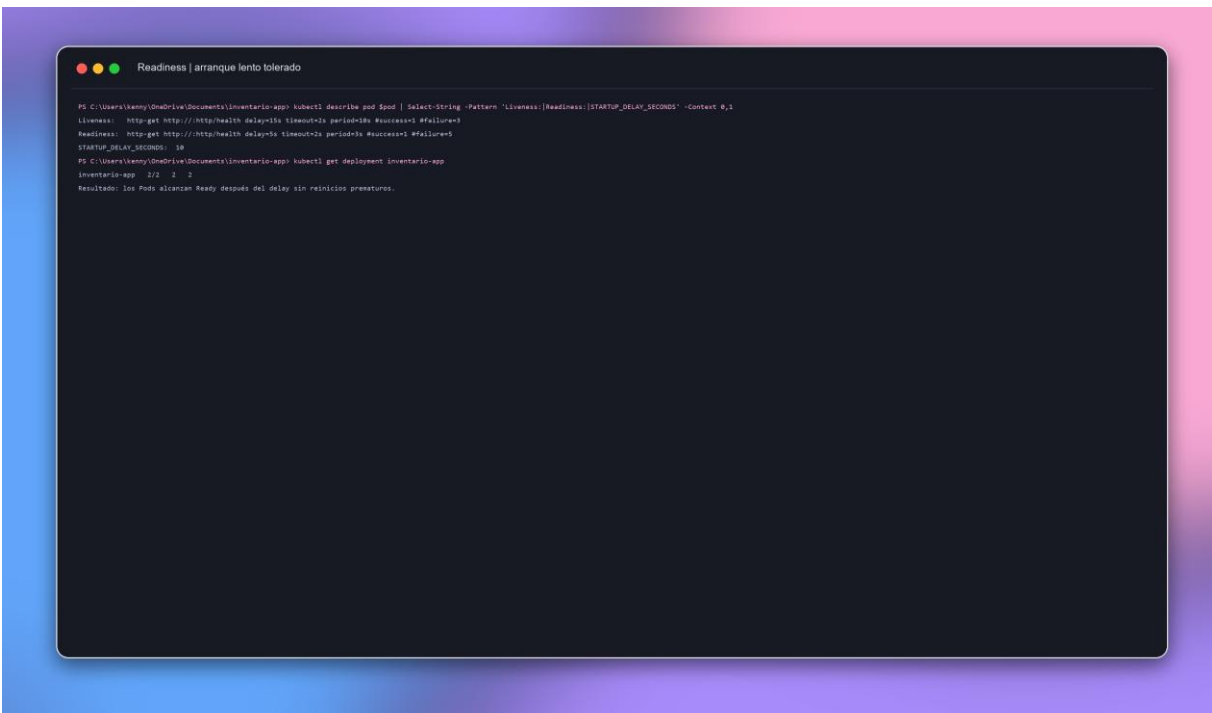
```
Kubernetes Secret | variable inyectada

PS C:\Users\keny\OneDrive\Documents\inventario-app> kubectl get pods -l app=inventario-app -o [jsonpath='{.items[0].metadata.name}']
PS C:\Users\keny\OneDrive\Documents\inventario-app> kubectl describe pod $pod | Select-String -Pattern 'API_KEY[[:inventory-secret]]' -Context 8,1
Environment:
  API_KEY:  cact to the key 'API_KEY' in secret 'inventory-secret': Optional: false
PS C:\Users\keny\OneDrive\Documents\inventario-app> curl desde Service: /version
{"version":"v1","color":"blue","secretConfigured":true,"secretMasked":"[REDACTED]"}
Resultado: el valor se obtiene por secretKeyRef y nunca se imprime sin máscara.
```

Nota. La variable `API_KEY` se consume mediante `secretKeyRef` sin exponer su valor en el Deployment ni en la respuesta de la aplicación.

Figura A11

Readiness durante el arranque lento

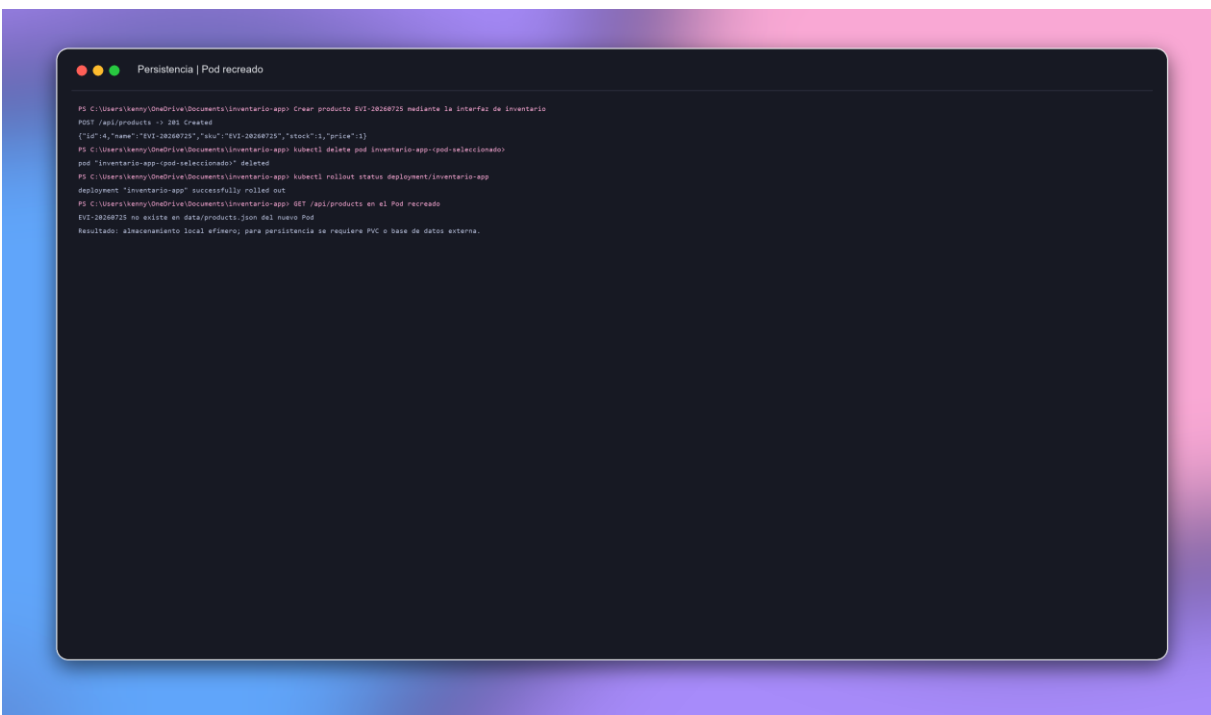


```
PS C:\Users\keny\OneDrive\Documents\inventario-app> kubectl describe pod $pod | Select-String -Pattern "Liveness:|Readiness:|STARTUP_DELAY_SECONDS" -Context 8,1
Liveness:  http-get http://http/health delay=1s timeout=1s period=10s #success=1 #failure=0
Readiness:  http-get http://http/health delay=1s timeout=1s period=10s #success=1 #failure=0
STARTUP_DELAY_SECONDS:  10
PS C:\Users\keny\OneDrive\Documents\inventario-app> kubectl get deployment inventario-app
inventario-app  3/2   2   2
Resultado: los Pods alcanzan Ready después del delay sin reinicios prematuros.
```

Nota. Kubernetes mantiene al Pod fuera del tráfico durante el retraso y lo incorpora cuando /health responde correctamente.

Figura A12

Pérdida del producto al recrear el Pod



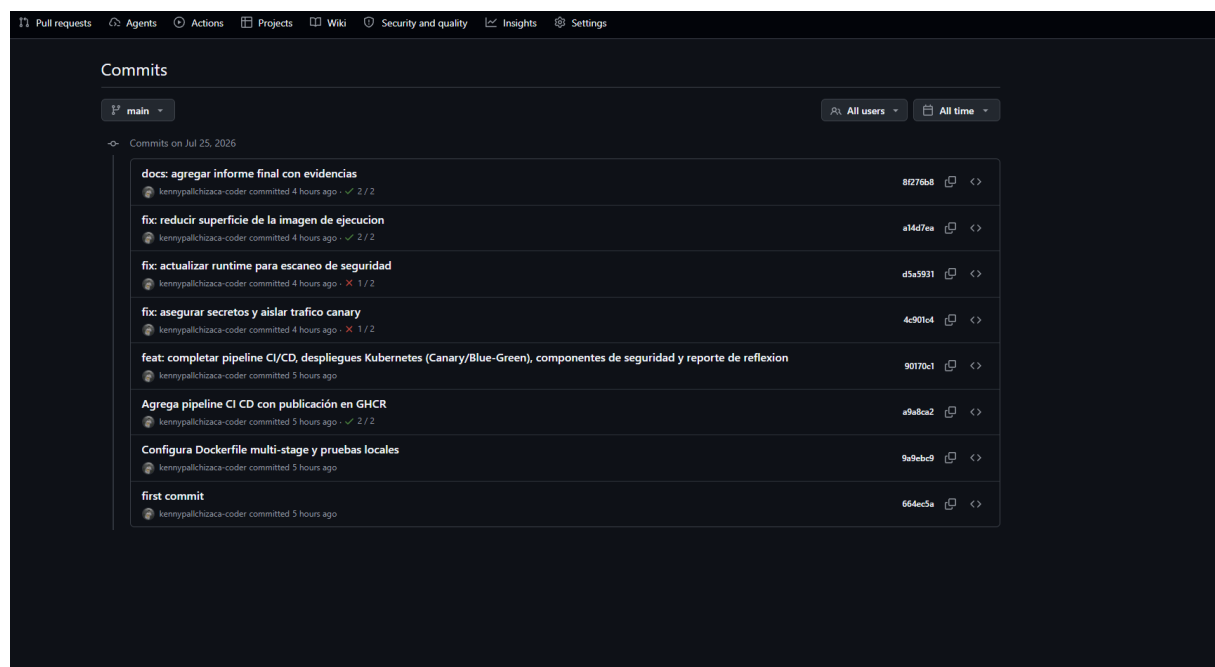
```
Persistencia | Pod recreado

PS C:\Users\keny\OneDrive\Documents\Inventario-app> Crear producto EVI-20200725 mediante la interfaz de Inventario
POST /api/producto -> 201 Created
{"id":"4","name":"EVI-20200725","sku":"EVI-20200725","stock":1,"price":1}
PS C:\Users\keny\OneDrive\Documents\Inventario-app> kubectl delete pod Inventario-app-(pod-seleccionado)
pod "Inventario-app-(pod-seleccionado)" deleted
PS C:\Users\keny\OneDrive\Documents\Inventario-app> kubectl rollout status deployment/Inventario-app
deployment "Inventario-app" successfully rolled out
PS C:\Users\keny\OneDrive\Documents\Inventario-app> GET /api/producto en el Pod recreado
EVI-20200725 no existe en data/products.json del nuevo Pod
Resultado: almacenamiento local efímero; para persistencia se requiere PVC o base de datos externa.
```

Nota. El producto creado no persiste después de recrear el Pod, comportamiento coherente con el uso de data/products.json en almacenamiento efímero.

Figura A13

Historial de commits y estado de las comprobaciones



Nota. La captura muestra la secuencia de cambios, dos ejecuciones que requirieron corrección (1/2) y ejecuciones posteriores exitosas (2/2).

Repositorio público: [inventario-app en GitHub](https://github.com/kenrypalcizaca/inventario-app)